

CalSoft, Deep Analysis of the Metrology Mathematics and Module Structure

Repository: cirqen-labs | Branch: improvement-plan | Reviewed: 17 September 2026

Findings 1-7, 13, 14 and 18 have since been fixed, see section 0.

Scope of review: all 49 Python modules of the CalSoft Django app (13,040 lines), plus the sync-table registration in `sync/config.py` that carries CalSoft rows between the field database and HQ. Every formula, threshold, classification rule and rounding step in the app was traced from data entry through to the printed certificate.

System boundary. Calibration does not end at this repository. The platform spans two separate git repositories, and the name `hq_server` refers to two unrelated codebases:

Path	Repository	Role
<code>cirqen-labs/hq_server/</code>	<code>cirqen-labs</code>	Software update distribution (Ed25519 package signing, ~1,500 lines)
<code>~/Desktop/hq_server/</code>	<code>wairegimaina/hq_server</code>	HQ sync and certificate authority (47 modules, 25,542 lines)

The second repository mints certificate numbers and flips sessions to approved. Section 7a records what that adds to the findings below; the formula-level findings in Section 7 concern CalSoft only. The HQ certificate and record-integrity paths are audited separately in `05_HQ_AND_SYNC_AUDIT.pdf`.

0. Which findings are closed

This audit records the state on 17 September. Implementation has since closed ten of the eighteen findings. The descriptions below are left as written because they document why each change was made; this table is the status.

#	Finding	State
1	Two verdicts; the certificate used the weaker	Fixed. <code>conformity_failed</code> reads strict conformity; the 40% rate is triage only
2	Error column has the opposite sign to convention	Open, decision needed. Both conventions are legitimate; the column, the stored data and the on-screen preview must be made to agree
3	Printed uncertainty budget does not add up	Fixed. Reference column printed; verified to reconcile on the page
4	Drift and linearity report confident wrong answers	Fixed. Both read what the engines return, report real numbers, and log when they cannot
5	<code>calculate_linearity</code> cannot be called successfully	Fixed. Called correctly from both the session page and the certificate
6	Uncertainty computed, printed, then ignored	Fixed. Guard banding applied, three outcomes, rule printed, TUR warned
7	Drift grades and interval advice are unit-blind	Fixed. Graded as a fraction of tolerance, on both the session page and the certificate

#	Finding	State
8	Quality assurance never runs on submissions	Open
9	Readings accepted below the declared count	Open, decision needed. Deriving k made the consequence visible: two readings now yield $k = 12.706$
10	Missing points corrupt the failure rate	Partly fixed. The verdict honours a failed session flag; the rate still counts only rows that exist
11	QR verification verifies nothing	Open. The largest remaining gap by weight
12	QR payload notation misleading and truncated	Open
13	Decimal context clobbered process-wide	Fixed. Duplicate module header removed; precision set once at 28
14	k=2 asserted unconditionally in the notes	Fixed. The notes quote the factor actually applied and state that it was derived
15	calculate_degrees_of_freedom returns a constant	Fixed. Real Welch-Satterthwaite; the old stub delegates to it
16	Structural debt (duplicate save, dead paths)	Partly fixed. Three functions reading a nonexistent field repaired; dead classes remain
17	No test covers any of the mathematics	Fixed. 181 tests, 95 with no database, every formula pinned
18	Two non-interoperating numbering algorithms	Fixed. Local allocation removed from all three paths; HQ is the sole allocator

Two of the open items are waiting on a decision rather than on work: finding 2 (sign convention) and finding 9 (minimum readings). Both change what stored data means or what the field may submit.

1. What CalSoft is

CalSoft is the calibration engine of the Equiper platform. It takes repeated manual readings of a device under test against a reference standard, reduces them to a mean and an uncertainty, decides pass or fail against a tolerance, and issues a numbered PDF certificate. It also keeps a parallel history table so it can regress drift across a device's calibration lifetime.

The app has four functional layers:

Layer	Modules	Responsibility
Data model	models.py (1,142 lines)	22 models; hosts the per-point calculation
Math core	utils.py (881 lines)	Statistics, uncertainty, drift, QA, metrology helpers
Orchestration	view_modules/ (14 modules)	Session capture, approval, API, imports
Reporting	pdf_generators/ (13 modules)	Certificate composed from 8 ReportLab mixins

2. The parts

2.1 Data model, the measurement chain

The schema encodes a five-level hierarchy, each level narrowing the measurement:

```
CalibrationProcedure      the method ("NIBP calibration")
  \- CalibrationParameter  what is measured (unit, tolerance, u_ref, k, n)
    \- SubParameter        a split axis (systolic / diastolic)
      \- SetValue          a test point (nominal value applied)
        \- CalibrationReading 10 reading slots + computed results
```

CalibrationSession is the event: one device, one procedure, one moment, one technician, one set of environmental conditions. SessionParameterResolution pins the instrument resolution *per session*, which is correct metrology, the display resolution belongs to the device being calibrated on the day, not to the procedure definition.

CalibrationReading is the arithmetic unit. It stores reading_1 ... reading_10 as ten discrete DecimalField(15,6) columns, then nine computed columns: mean, standard_deviation, error, type_a_uncertainty, type_b_uncertainty, reference_uncertainty_component, combined_uncertainty, expanded_uncertainty, passes_tolerance.

HistoricalCalibration is a denormalised flattening of every computed reading, keyed by device_serial + parameter_name + set_value + calibration_date. It exists solely so drift can be regressed without joining five tables.

2.2 Math core, utils.py

Eight top-level units:

Unit	Purpose	Live?
statistics_from_readings()	mean, sample s, n	unused
calculate_uncertainties()	full budget, float	unused
CalibrationCalculator	the live per-point engine	yes
DriftAnalyzer	regression over HistoricalCalibration	called, output discarded
ReportGenerator	HTML certificate + budget report	unused
QualityAssurance	outliers, variation, trend, capability	API only
TrendAnalysis	monthly pass-rate / uncertainty slope	broken call site
CalibrationValidator	procedure and session completeness	unused
MetrologyUtils	TUR, interval, guard band, k, dof	entirely unused

2.3 Reporting, the certificate

BtwelveHospitalCertificateGenerator composes eight mixins: StylesMixin, ResultsMixin, QrMixin, HeaderMixin, SectionsMixin, AnalysisMixin, SignatureMixin, DriftMixin. The mixin split is clean, but every one of the thirteen files repeats the same 35-line import block, including numpy, matplotlib, qrcode and twelve ReportLab symbols each module does not use.

3. The mathematics, step by step

3.1 Central tendency and dispersion

`CalibrationCalculator.calculate_statistics (utils.py:121)` takes the readings present, coerces each through `Decimal(str(r))`, and computes

```
n      = count of non-null readings
x_bar  = (1/n) | sum x_i
s      = sqrt( sum(x_i - x_bar)^2 / (n - 1) )      sample standard deviation
```

Bessel's correction ($n-1$) is correct: s estimates the population sigma from a sample, which is what Type A evaluation requires. The function returns `None` when $n < 2$, because s is undefined at $n = 1$.

3.2 Type A, repeatability

`calculate_type_a_uncertainty (utils.py:144):`

```
u_A = s / sqrt(n)
```

This is the standard uncertainty of the *mean*, not of a single indication. It is the right quantity, because the reported value is x_{bar} . Correct per JCGM 100:2008 section 4.2.3.

3.3 Type B, finite display resolution

`calculate_type_b_uncertainty (utils.py:156):`

```
u_res = resolution / sqrt(12)
```

This is the standard deviation of a rectangular distribution of full width `resolution`, i.e. $a/\sqrt{3}$ with half-width $a = \text{resolution}/2$. Correct for a display whose last digit quantises to the nearest count.

Note the interpretation this encodes: `resolution` is taken as the **full quantisation step**, not the half-interval. If a technician enters the half-interval, the uncertainty is understated by a factor of two. Nothing in the UI or the field help text states which convention applies.

3.4 Reference standard contribution

`calculate_reference_uncertainty (utils.py:166)` divides by k when the stored figure is expanded:

```
u_ref = U_ref / k      (reference certificate quotes expanded U at k=2)
```

The `CalibrationParameter.reference_uncertainty` field carries `help_text="k=2"` so the stored value is expanded by contract, and dividing is correct.

3.5 Combination

`calculate_combined_uncertainty (utils.py:180):`

```
u_c = sqrt( u_A^2 + u_res^2 + u_ref^2 )
```

Root-sum-square of three components treated as independent. Reasonable, the device's own repeatability, its display quantisation and the standard's accredited uncertainty are genuinely uncorrelated, so all covariance terms

vanish. Sensitivity coefficients are implicitly 1, valid only because every component is already expressed in the measurand's own units. That holds here but it means the model cannot represent any influence quantity needing a sensitivity coefficient, temperature coefficient, hydrostatic head, lead resistance.

3.6 Expansion

calculate_expanded_uncertainty (utils.py:193):

```
U = k | u_c          k from CalibrationParameter.coverage_factor, default 2.0
```

3.7 Error and the conformity decision

In CalibrationReading.calculate_statistics (models.py:645):

```
error = set_value - mean
passes_tolerance = |error| <= tolerance
```

Tolerance resolves sub-parameter first, then parameter (models.py:674).

The decision rule is a bare tolerance comparison. The measurement uncertainty that the app just spent five steps computing plays **no part** in the pass/fail call. There is no guard band, no shared-risk statement, no acceptance-limit reduction. A reading at error = tolerance - 0.000001 with $U = 3 \times \text{tolerance}$ is recorded as an unqualified PASS.

MetrologyUtils.calculate_guard_banding exists in the codebase and implements exactly the missing rule, but nothing calls it.

3.8 Session verdict

_process_readings (view_modules/calibration.py:232-280) folds the per-point results with a logical AND:

```
overall_pass = AND over all expected points of passes_tolerance
               AND no expected point was left blank
```

This is strict conformity, one failed point fails the session.

3.9 The certificate verdict, a second, different rule

certificate.py:95 computes an independent verdict:

```
failure_rate = failed_readings / total_readings
is_failed_report = failure_rate >= 0.40
```

These two rules disagree across a wide band. A device failing 2 of 6 points (33%) is overall_pass = False in the database and, on the certificate, STATUS: PASSED, in the QR payload, in the header, and in note 5. See Finding 1.

4. Worked example, the full chain

Parameter: NIBP systolic. Nominal 200 mmHg, tolerance ± 3 mmHg, reference standard $U = 0.25$ mmHg ($k=2$), session resolution 1 mmHg, $k = 2$. Five readings: 201, 202, 201, 203, 202.

```

n      = 5
x_bar  = 1009 / 5                      = 201.800000
s      = sqrt(2.8 / 4)                 = 0.836660 -> 0.836660 (6 dp)
u_A    = 0.836660 / sqrt(5)           = 0.374166 -> 0.374166
u_res  = 1 / sqrt(12)                 = 0.288675 -> 0.288675
u_ref  = 0.25 / 2                     = 0.125000
u_c    = sqrt(0.374166^2 + 0.288675^2 + 0.125^2)
      = sqrt(0.140000 + 0.083333 + 0.015625)
      = sqrt(0.238958)                 = 0.488834 -> 0.488834
U      = 2 × 0.488834                 = 0.977668 -> 0.977668
error  = 200 - 201.8                  = -1.800000
|-1.8| <= 3                          -> PASS

```

Reported on the certificate: 200 | 201.8000 | 0.8367 | -1.8000 | 3.000000 | PASS and in the uncertainty table 200 | 0.3742 | 0.2887 | 0.4888 | 0.9777 | 2.0.

Three things an assessor would immediately query in that output:

1. $0.3742^2 + 0.2887^2 = 0.2234$, whose square root is 0.4726, not the printed 0.4888. The reference component 0.1250 reconciles it, but the table has no column for it. The printed budget does not add up (Finding 3).
2. The device reads 1.8 mmHg **high**, yet the Error column shows -1.8000 (Finding 2).
3. $U = 0.98$ against a tolerance of 3.00 gives $TUR = 3.07:1$, below the 4:1 floor that the app's own MetrologyUtils docstring names, and nothing warns (Finding 6).

5. The secondary mathematics

5.1 Linearity

Two implementations exist; neither reaches a certificate.

`CalibrationCalculator.calculate_linearity` (utils.py:207) is a proper least-squares fit with residuals expressed as a percentage of full scale:

```

slope      = (n | sumxy - sumx | sumy) / (n | sumx^2 - (sumx)^2)
intercept  = (sumy - slope | sumx) / n
residual   = y_i - (slope | x_i + intercept)
err_pct    = residual / full_scale × 100      full_scale defaults to max(x) - min(x)

```

It is called once, at utils.py:399, as `CalibrationCalculator.calculate_linearity(set_values, errors)`, unbound, with `set_values` binding to `self`. That call raises `TypeError` every time (Finding 5). It also passes `errors` where the signature wants

measured values.

`_calculate_linearity_analysis` (view_modules/sessions.py:215) is the version the session-detail page actually uses. It calls `TrendAnalysis.calculate_linear_regression(...)`, a method that does not exist on `TrendAnalysis`. The `AttributeError` is swallowed by a bare `except Exception` so every parameter silently returns `{"error": "Unable to calculate linearity"}` (Finding 4).

The certificate's "Linearity Analysis" section (analysis.py:87) is therefore not analysis at all: it plots set value against mean with matplotlib and draws no fit line, no residuals and no computed linearity error.

5.2 Drift

Two independent drift engines read two different data sources.

DriftMixin._compute_drift_metrics (drift.py:129), the one on the certificate, regresses stored error against elapsed days across previous CalibrationSession rows for the same serial:

```
days_i      = (session_date_i - session_date_0) in days
slope       = (n | sum(d | e) - sumd | sume) / (n | sumd^2 - (sumd)^2)    error units per day
per_year    = slope × 365
R^2         = 1 - SS_res / SS_tot      clamped at 0, forced to 1.0 when SS_tot = 0
total_drift = e_last - e_first
```

Stability is then bucketed on the absolute yearly rate:

```
|per_year| < 0.1  -> Excellent
          < 0.5  -> Good
          < 1.0  -> Fair
          else   -> Poor
```

Those four thresholds are **dimensionless constants applied to a dimensioned quantity**. A 0.4 mmHg/year pressure drift and a 0.4 mV/year ECG drift and a

0.4 °C/year temperature drift all grade "Good", and a flow parameter measured in mL/min grades "Poor" at 1 mL/min/year, which is excellent for an infusion pump. The grade carries no information unless every parameter in the estimate shares a unit and a magnitude. It does not (Finding 7).

The same constants drive the certificate's printed recommendation:

```
avg|per_year| < 0.5  -> "Calibration interval is appropriate."
              < 1.0  -> "Consider shortening calibration interval."
              else   -> "Shorten calibration interval urgently."
```

So the certificate tells the hospital to shorten a service interval on the basis of a unit-blind comparison. That recommendation is printed in a coloured bar, above the technician's signature.

Uncertainty trend uses a split-half ratio with fixed ±20% bands:

```
half = max(n // 2, 1)
Increasing if mean(U[half:]) > 1.2 | mean(U[:half])
Decreasing if mean(U[half:]) < 0.8 | mean(U[:half])
else Stable
```

With n = 2 (the common case for a device on its second calibration) the split is one point versus one point, and a single noisy session flips the verdict.

DriftAnalyzer.analyze_drift (utils.py:254) is the second engine. It reads HistoricalCalibration, groups by parameter and set value, and returns a nested {param: {set_value: metrics}} dict. Its one call site (view_modules/sessions.py:126) reads drift_data.get("trend"), .get("recommendation"), .get("stability_index"), .get("confidence_level"), .get("parameter_drifts", []), five keys the function never produces. Every lookup falls through to its default, so the session-detail page displays the literal string **"No significant drift detected"** and an empty parameter list for every device, whatever the data says (Finding 4).

5.3 Failure statistics

calculate_failure_statistics (results.py:324) counts per parameter and overall, then flags any parameter at or above 40%. analysis.py:170 grades the recommendation:

```
>= 0.80 -> "Critical - Immediate repair required"
>= 0.60 -> "Major issue - Service needed"
else     -> "Minor issue - Monitor closely"
```

Two structural problems. First, the denominator is *readings that exist*. A point left blank by the technician sets overall_pass = False but creates no CalibrationReading row, so it is absent from both numerator and denominator, an incomplete session can show a 0% failure rate. Second, the failure rate is unweighted across parameters, so a procedure with one parameter at 12 points and another at 2 lets the 12-point parameter dominate the verdict entirely.

5.4 Outlier detection

QualityAssurance.validate_readings (utils.py:476) uses the modified Z-score:

```
MAD = median(|x_i - median(x)|)
M_i = 0.6745 | (x_i - median) / MAD      flag when |M_i| > 3.5
```

The constant 0.6745 and the 3.5 threshold are the standard Iglewicz-Hoaglin values, and MAD is the right robust scale for $n = 5$. Two implementation defects: the median is taken as `sorted(x)[n//2]`, which is the upper of the two central values for even n rather than their average; and `PDFConfig.UNCERTAINTY_CONFIG` declares 'outlier_method': 'grubbs', which is not what is implemented and is never read by anything.

The function also flags any monotonic run of ≥ 5 readings as a warm-up trend. With `num_readings = 5`, the model default, a monotonic sequence has probability $2/5! = 1/60$ by chance, so this will fire occasionally on clean data.

More importantly: this whole function runs **only** from the `api_validate_readings` endpoint. The submission path (`_process_readings`) never calls it. No outlier check, no variation check and no trend check gates a real calibration.

5.5 The unreached metrology layer

MetrologyUtils has no call sites anywhere in the repository. It contains the four pieces of reasoning the app most visibly lacks:

```
TUR          = tolerance / uncertainty                (4:1 floor)
guard_band    = 2 | U; effective_tolerance = tolerance - guard_band
interval_days = (tolerance / z) / |drift_rate|          clamped to [30, 1095]
k(confidence) = lookup {68.27:1.0, 95:1.96, 95.45:2.0, 99:2.576, 99.73:3.0}
```

calculate_degrees_of_freedom is declared as a Welch-Satterthwaite implementation and its body is `return max(2, 10)`, a literal 10 with no dependence on either argument. Nothing calls it, which is fortunate.

6. Numerical behaviour

6.1 Decimal precision is silently halved

utils.py is two modules concatenated. Line 10 sets `getcontext().prec = 28`; line 104, a second `# utils.py` header with a second import block, resets it to 12 and comments it "high precision". The later assignment wins at import time and applies process-wide to every Decimal operation in the application, including other apps.

6.2 Quantisation cascades through the budget

calculate_statistics in models.py rounds each intermediate to 6 dp with ROUND_HALF_UP and feeds the rounded value into the next step:

```
s rounded -> u_A computed from rounded s -> u_c from rounded u_A -> U from rounded u_c
```

Four sequential roundings. At 1e-6 granularity this is immaterial for mmHg but for a parameter in amperes or mV it is not: a resolution of 0.0001 gives $u_{res} = 0.0000289$, which rounds to 0.000029, a 0.4% shift, and prints on the certificate, formatted to 4 dp, as 0.0000.

6.3 Decimal is abandoned mid-calculation

calculate_combined_uncertainty is written in Decimal but its body is `Decimal(str(math.sqrt(u_A2 + u_res2 + u_ref**2)))`. `math.sqrt` coerces to float64, so the RSS step, the one operation in the budget where precision could matter, runs in binary floating point and is then re-wrapped. The same happens in `calculate_type_a_uncertainty` via `math.sqrt(count)`.

6.4 The API and the database disagree

api_calculate_uncertainty (view_modules/api.py:332) computes u_A from the

unrounded stats["std_dev"], while models.py computes it from the

6-dp-rounded self.standard_deviation. The live preview a technician sees while typing and the value stored on submit are computed from different inputs and can differ in the last digits. The endpoint also bypasses `calculate_reference_uncertainty` and divides by k inline, so the `reference_is_expanded` flag is unreachable from either path.

6.5 Certificate formatting truncates below the reported resolution

All six uncertainty columns print with :.4f (results.py:296-300) against values stored at 6 dp. Any uncertainty component below 0.00005 prints as 0.0000. For fine-resolution parameters the certificate's uncertainty table is a column of zeros.

7. Findings

Ordered by consequence. File references are to the current working tree.

1, Two different pass/fail rules, and the certificate uses the weaker one. certificate.py:95.

session.overall_pass (all points must pass) never appears in any pdf_generators module; the printed verdict, the header banner, note 5 and the QR STATUS field all derive from `failure_rate >= 0.40`. A device failing up to 39% of its test points is certified PASSED while the database records it as failed. This is the most serious finding: it puts a non-conforming device back into clinical service with a clean certificate.

2, The Error column has the opposite sign to the metrological convention. models.py:645 computes `error = set_value - mean`. Error of indication is defined as indication minus reference, so this quantity is the *correction*, not the error. The certificate labels it "Error". A device reading high shows a negative number. Drift regression inherits the inverted sign, so every "Increasing " direction arrow on the certificate points the wrong way.

3, The printed uncertainty budget does not add up. results.py:290 builds the table as Set Value | Type A | Type B | Combined | Expanded | k . `reference_uncertainty_component` is stored but never printed, so `Combined != sqrt(TypeA^2 + TypeB^2)` on the page. An ISO/IEC 17025 assessor checking the arithmetic finds a discrepancy with no stated cause.

4, Both drift and linearity report confident wrong answers, silently. `view_modules/sessions.py:126` reads five keys `DriftAnalyzer.analyze_drift` never returns, so the session page prints "No significant drift detected" for every device regardless of the data. `sessions.py:228` calls `TrendAnalysis.calculate_linear_regression`, which does not exist; the `AttributeError` is caught by a bare except `Exception` and every parameter degrades to "Unable to calculate linearity". Neither failure is logged at error level in a way that surfaces. Both look like working features.

5, calculate_linearity cannot be called successfully. `utils.py:399` invokes an instance method unbound, so `set_values` binds to `self` and `measured_values` is never supplied, `TypeError` on every call. It also passes errors where measured values are expected. The only real linearity implementation in the codebase has never run.

6, Measurement uncertainty is computed, printed, and then ignored in the decision. `models.py:680`. `passes_tolerance = |error| <= tolerance`, with no guard band and no TUR check. `MetrologyUtils.calculate_guard_banding` and `calculate_measurement_capability` implement both and are dead code. For a hospital laboratory this is the gap between "we report uncertainty" and "we use it", and it is the gap accreditation assessors probe first.

7, Drift stability grades and interval recommendations are unit-blind. `drift.py:165-172` and `drift.py:343`. Four bare constants (0.1/0.5/1.0) classify a dimensioned rate, and the same constants generate a printed recommendation to change a calibration interval. The grade is meaningless across parameters of different units and magnitudes, and the certificate prints it as advice.

8, Quality assurance never runs on real submissions. `QualityAssurance.validate_readings`, `CalibrationValidator.validate_procedure` and `validate_calibration_data` are reachable only from `api_validate_readings` (one of them) or from nowhere (the other two). `_process_readings` calls none of them. Outliers, excessive variation, warm-up trends and incomplete procedures all pass through unchallenged.

9, Readings are accepted below the declared count with no signal. `view_modules/calibration.py:270` slices `readings[:parameter.num_readings]` but never checks the length from below. `calculate_statistics` proceeds at `n >= 2`, `num_readings` validates at `>= 3`, `PDFConfig` declares `minimum_readings: 3`, and `validate_calibration_data` requires 3, four different minimums, with the loosest one governing. A parameter declaring 10 readings certifies on 2, at roughly 2.2× the Type A uncertainty it should have, with nothing on the certificate indicating that `n` was 2.

10, Missing points corrupt the failure rate. An expected point with no readings sets `overall_pass = False` and creates no row. `total_readings` in `calculate_failure_statistics` counts rows, so the missing point is invisible to the rate. An incomplete session can print a 0% failure rate.

11, QR "authenticity verification" verifies nothing. `verification.py:195` returns `'valid': True` unconditionally, with the comment "This would be determined by database lookup". `generate_verification_url` base64-encodes `cert:session_id:timestamp` with no HMAC, so the token is forgeable by anyone who can read one certificate. Note 4 on every certificate reads "Certificate authenticity can be verified by scanning the QR code." `PDFConfig` records `'digital_signature': False`.

12, QR payload notation is misleading and silently truncated. `qr.py:108` emits `{set}{unit}>{mean}±{error}`. The $\pm$ conventionally introduces uncertainty; here it introduces the error. `results_data[:15]` drops every point past the fifteenth with no marker, so a 20-point procedure's QR silently omits a quarter of its results.

13, Decimal context is clobbered process-wide. `utils.py:104` resets `getcontext().prec` from 28 to 12 as a side effect of a duplicated module header, affecting every app in the project.

14, k=2 is asserted unconditionally in the certificate notes. `sections.py:270` prints "calculated using a coverage factor k=2" while the `k` column beside it renders `parameter.coverage_factor`, which is a per-parameter `DecimalField`. Set a parameter to `k=3` and the certificate contradicts itself on the same page.

15, calculate_degrees_of_freedom returns a constant. `utils.py:851`. Declared as Welch-Satterthwaite; body is `return max(2, 10)`. Unused, but it will mislead the next reader who reaches for it, and effective degrees of freedom is precisely what a defensible k requires.

16, Structural debt. `CalibrationParameter.save` is defined twice (`models.py:329` and `:333`); the first is unreachable. `HistoricalCalibration.save` sets `self.needs_sync`, a field the model does not declare, a silent no-op (sync still works, because `sync/config.py` drives it from Debezium CDC on the table, but the model is the only syncable CalSoft model missing the field). `CalibrationReading.set_reading` issues a full `save()` per reading, so a 10-point capture costs 10 UPDATES plus one more from `calculate_statistics`. The fake `SetValue` stand-in at `calibration.py:256` (`type('obj', (object,), ...)`) is passed to `objects.create(set_value=sv)` and cannot succeed. The `hasattr(reading, 'passes_tolerance')` fallbacks at `results.py:170` and `results.py:345` are unreachable, because the attribute is a model field and is always present.

17, No test covers any of the mathematics. Across `tests.py`, `test_calibration_flow.py` and `test_imports.py` (564 lines, 33 tests) there is not one assertion on a mean, a standard deviation, an uncertainty component, a tolerance decision or a drift rate. `tests.py:141` tests linearity by *mocking out* the regression that is itself broken, so the suite passes while the feature does not work. Every finding above would have been caught by a single test per formula.

7a. The certificate authority, across both repositories

The findings above treat certificate issuance as ending in CalSoft. It does not. `~/Desktop/hq_server` polls for approved sessions, allocates the certificate number and flips session status. That adds one finding, and corrects an impression left by section 9 below.

18, Two non-interoperating certificate-number algorithms share one prefix and one column.

	Algorithm	Serialized by	Database
HQ (<code>certificate_normalizer.py</code> <code>get_next_number</code> , called with <code>reuse_gaps=True</code> at <code>certificate_service_extra.py:231</code> and <code>certificate_service_base.py:428</code>)	Lowest missing number first, else $\text{max}+1$	<code>pg_advisory_xact_lock</code>	HQ
CalSoft (<code>models.py:495</code> + <code>calSchedules/grouping.py:178</code>)	Strictly $\text{max}+1$, never reuses	<code>select_for_update</code>	Local field

Both are individually sound. Each serialises correctly against concurrent callers *on its own database*. Neither lock is visible to the other, so the two can allocate independently, and `certificate_number` carries `unique=True`, so a collision surfaces as a sync conflict rather than a clean, early error.

The intended rule is that HQ allocates every number. The code honours it in the offline branch, which assigns no number at all (`certificate_number = None`, status `approved_pending_certificate`, a queued `PendingCertificate`). The online branch violates it: `is_hq_online()` is only an HTTP health probe, and `Equiper/db_router.py` returns "default" for every write, so a reachable HQ still results in the field site allocating `local_max + 1` in its own database. With several field sites, HQ issues per-device API keys, two sites at similar local maxima allocate the same number.

This is not hypothetical. `sync/cert_conflict_guard.py` exists to repair exactly this, names the production incident it was written for (**BNH-0093**), and supersedes two hand-written diagnostic scripts. Its repair reassigns the losing session's certificate number, which, if that certificate was already printed, silently desynchronises the filed

document from the record. 05_HQ_AND_SYNC_AUDIT.pdf section 3 covers the mechanism and the consequence.

What was checked and found sound. Three things that look like defects from a CalSoft-only reading are not, and should not be re-raised:

- **Gap reuse is deliberate and tested.** `tests/test_cert_numbering.py::test_gap_is_reused` asserts it as intended behaviour. It is a design decision, not an oversight.
- **Soft delete does not release a number.** `soft_delete_handler.py` flags `active_status` / `pending_delete` rather than removing rows, and no hard DELETE of sessions exists anywhere in either repository. Because `_fetch_existing_numbers` applies **no** status filter, a soft-deleted certificate's number remains in the "existing" set and stays out of the reusable pool. The absent filter is what makes reuse safe here; it is not a bug.
- **Wholesale renumbering is guarded.** `normalize_all_certificates` renumbers every certificate, but is reachable only through an endpoint requiring an explicit `{"dry_run": false, "confirm": true}`.

A correction to section 9 below. section 9 credits CalSoft for minting certificate numbers inside a locked transaction, and calls it more careful than the sibling modules. That remains true of the code as written, but it is not the whole mechanism: in normal online operation HQ is the allocator. The praise belongs to both implementations, and the criticism belongs to the fact that there are two.

On HQ's test coverage. HQ carries 41 tests across 25,542 lines, a lower density than CalSoft, but they are aimed at the things that can corrupt data: certificate numbering, certificate delivery, soft-delete cascade, last-write-wins upsert, null propagation, API keys, client-registration idempotency. The contrast with finding 17 is worth stating plainly: **HQ tests the allocation of the certificate number, while CalSoft tests none of the measurement mathematics printed on the certificate.** The platform has been more careful with the identifier than with the measurement.

8. What is absent relative to GUM / ISO 17025

Beyond the defects, four capabilities the model cannot currently express:

- **Sensitivity coefficients.** All components are assumed to be in the measurand's units with $c_i = 1$. No influence quantity requiring a coefficient, temperature coefficient, lead resistance, hydrostatic head,

can enter the budget.

- **Effective degrees of freedom.** k is a user-entered constant, not derived from `nu_eff`. With $n = 3-5$ readings, a 95% interval strictly needs t (2.78 at $nu = 4$), not 2.0. The app understates u by ~40% in the small- n case it most commonly operates in.
- **Correlation.** The RSS assumes independence with no facility to declare a covariance, which matters when one standard supplies several points.
- **Decision rules.** No guard band, no acceptance-limit reduction, no shared-risk statement, no declaration of the conformity rule used, required by ISO/IEC 17025:2017 section 7.8.6 when a statement of conformity is issued, which is exactly what this certificate issues.

9. What is sound

The critique above is dense, so it is worth stating plainly what holds up.

The five formulas at the core, Bessel-corrected s , $u_A = s/\sqrt{n}$, $u_{res} = r/\sqrt{12}$, $u_{ref} = U/k$, and RSS combination, are each individually correct and correctly implemented. The choice to pin resolution per session rather than per procedure is right and is not the obvious design. `Decimal` throughout the storage layer with explicit

ROUND_HALF_UP quantisation is the correct instinct for a metrology record. The modified Z-score with MAD is the right robust outlier test for $n = 5$, with the right constants. The drift regression, R^2 included, is correctly derived. The certificate-number minting uses `select_for_update` inside a transaction, which is the correct way to serialise concurrent issuance. The mixin decomposition of the PDF generator and the `view_modules` split are both good structure. `backfill_calibration_stats` is an exemplary repair command: it documents the bug it fixes, recomputes dependents, backfills the history table and supports `--dry-run` via a sentinel rollback.

The problem is not the mathematics. The problem is that the mathematics is surrounded by four parallel half-wired implementations, and the layer that prints the verdict does not consult the layer that computed it.

10. Recommended order of work

Now, correctness of the issued document

1. Make the certificate verdict `session.overall_pass`. Delete the 40% rule from the verdict path; keep the rate as a maintenance-triage signal only.
2. Fix the error sign convention to mean - `set_value`, migrate stored rows

and re-derive drift directions. Or keep the sign and relabel the column "Correction", but choose, and state it on the certificate.

1. Add the reference-uncertainty column to the printed budget table so it reconciles.
2. Print the actual coverage_factor in note 3, and print n beside each point.
3. Widen uncertainty formatting from `:.4f` to significant figures, or to the stored 6 dp.

Next, make the decision defensible

1. Wire `MetrologyUtils.calculate_guard_banding` into `passes_tolerance`, and print the decision rule on the certificate.
2. Wire `calculate_measurement_capability` and warn below TUR 4:1.
3. Replace the unit-blind drift grades with a tolerance-relative measure,

`|drift_per_year| / tolerance` is dimensionless and comparable.

1. Either implement `nu_eff` and derive k , or state on the certificate that $k = 2$ is used with an assumed normal distribution and large nu .

Then, remove the ambiguity

1. Delete or fix the dead parallel implementations: `calculate_uncertainties`, `statistics_from_readings`, `ReportGenerator`, `CalibrationValidator`, the unbound `calculate_linearity` call, and the two broken call sites in `sessions.py`. One implementation per formula.
2. Split `utils.py` at line 101 into two modules and restore `prec = 28` once.
3. Call `QualityAssurance.validate_readings` from `_process_readings`, and enforce `num_readings` from below.
4. Implement real QR verification, HMAC the payload, look the certificate up

and return a real `valid`, or remove note 4 from the certificate.

1. Replace bare except `Exception:` `pass` in the analysis paths with `logged`, surfaced failures.

Alongside all of it

1. One unit test per formula, asserting against hand-computed values. The worked example in section 4 is a ready first test vector. This is the single change that prevents findings 1-7 recurring.

Prepared from static analysis of the working tree at branch improvement-plan. No code was modified. Line references may shift as the tree changes.

